

Reviewer Pack — Minimal Order of Geometric Invariant Theory and Circuit Mono...

Entry f5c9b3eb3fcd · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 19:07:34 UTC

Minimal Order of Geometric Invariant Theory and Circuit Monotone Width

Entry ID: f5c9b3eb3fcd **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 19:07:34 UTC

1. Conjecture

Field A (mathematical branch): Geometric Invariant Theory **Field B** (complexity object): Boolean Circuit Complexity (Circuit Monotone Width)

Statement:

For every CNF φ with n variables, the minimal order of the geometric invariant group for the moduli space of tropical curves defined by φ is linearly correlated with its circuit monotone width $w(\varphi)$, such that $\min_order(GI(\varphi)) = \Theta(w(\varphi))$.

Rationale (proposer's reasoning):

Geometric Invariant Theory provides a framework for studying symmetries of algebraic varieties, which could reveal hidden structural properties in the complexity of computing satisfying assignments for Boolean formulas. The minimal order of the geometric invariant group measures the degree of symmetry, and if this were correlated with circuit monotone width, it might suggest that certain symmetries are more common in hard instances.

Taxonomy category: geometric_invariant_theory_to_circuit_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 524074b7b4e65deb

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient (r) between $\min_order(GI(\varphi))$ and $w(\varphi)$ for all CNFs φ with n variables is ≥ 0.7 , where $n \leq 40$, across at least 30 seeds. The criterion is falsified if $r < 0.5$ or any seed produces a correlation coefficient outside the range $[0.2, 1.0]$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: ADJACENT_OK against 4 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - intitle:Geometric Invariant Theory AND circuit monotone width - title:Minimal Order of Geometric Invariant Theory OR tropical curves AND monotone width - author:(Corollary OR Theorem) AND (Geometric Invariant Theory OR circuit monotone width)

Top relevant hits considered: - [http://arxiv.org/abs/2311.04204v3] Sharp Thresholds Imply Circuit Lower Bounds: from random 2-SAT to Planted Clique - [http://arxiv.org/abs/2108.04557v3] Brauer diagrams, modular operads, and a graphical nerve theorem for circuit algebras - [http://arxiv.org/abs/2412.20262v4] Modular operads, iterated distributive laws and a nerve theorem for circuit algebras - [http://arxiv.org/abs/1305.0926v3] Geometric Invariant Theory and Roth's Theorem

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        cnf = []
        for _ in range(10): # Generate 10 clauses with n variables each
            clause = [random.randint(-n, -1), random.randint(1, n)]
            cnf.append(clause)
        return cnf

    def circuit_monotone_width(cnf):
        # Placeholder implementation for monotone width calculation
        # Replace this with actual implementation if available
        return len(cnf) # Simplified example

    def geometric_invariant_group_order(cnf):
        # Placeholder implementation for geometric invariant group order calculation
        # Replace this with actual implementation if available
        return len(cnf) # Simplified example
```

```

n = random.randint(5, 40)
cnf = generate_cnf(n)
min_order = geometric_invariant_group_order(cnf)
w_phi = circuit_monotone_width(cnf)

return {
    "metric_name": "min_order(GI( $\phi$ ))",
    "metric_value": min_order,
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": False,
    "counterexample": "mapping_undefined"
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 67, 71, 73, 79, 83, 89, 97]
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction=1.0")
    elif any(not result["conjecture_holds"] for result in results) and support_fraction >= 0.8:
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={seeds[results[0]['n_max']-1]}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

ame': 'min_order(GI( $\phi$ ))', 'metric_value': 10, 'instances_tested': 1, 'n_max': 21, 'conjecture_holds': False
TRIAL: {'metric_name': 'min_order(GI( $\phi$ ))', 'metric_value': 10, 'instances_tested': 1, 'n_max': 18, 'conjecture_holds': False}
TRIAL: {'metric_name': 'min_order(GI( $\phi$ ))', 'metric_value': 10, 'instances_tested': 1, 'n_max': 15, 'conjecture_holds': False}
TRIAL: {'metric_name': 'min_order(GI( $\phi$ ))', 'metric_value': 10, 'instances_tested': 1, 'n_max': 7, 'conjecture_holds': False}
TRIAL: {'metric_name': 'min_order(GI( $\phi$ ))', 'metric_value': 10, 'instances_tested': 1, 'n_max': 25, 'conjecture_holds': False}
TRIAL: {'metric_name': 'min_order(GI( $\phi$ ))', 'metric_value': 10, 'instances_tested': 1, 'n_max': 23, 'conjecture_holds': False}
TRIAL: {'metric_name': 'min_order(GI( $\phi$ ))', 'metric_value': 10, 'instances_tested': 1, 'n_max': 9, 'conjecture_holds': False}
TRIAL: {'metric_name': 'min_order(GI( $\phi$ ))', 'metric_value': 10, 'instances_tested': 1, 'n_max': 38, 'conjecture_holds': False}
TRIAL: {'metric_name': 'min_order(GI( $\phi$ ))', 'metric_value': 10, 'instances_tested': 1, 'n_max': 16, 'conjecture_holds': False}
TRIAL: {'metric_name': 'min_order(GI( $\phi$ ))', 'metric_value': 10, 'instances_tested': 1, 'n_max': 36, 'conjecture_holds': False}
TRIAL: {'metric_name': 'min_order(GI( $\phi$ ))', 'metric_value': 10, 'instances_tested': 1, 'n_max': 19, 'conjecture_holds': False}
TRIAL: {'metric_name': 'min_order(GI( $\phi$ ))', 'metric_value': 10, 'instances_tested': 1, 'n_max': 17, 'conjecture_holds': False}
RESULT: INCONCLUSIVE insufficient_data

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test code uses placeholder implementations for calculating the geometric invariant group order and circuit monotone width, which are not faithful to the mathematical definitions. This could lead to incorrect results.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code uses placeholder implementations for calculating the geometric invariant group order and circuit monotone width, which are not faithful to the mathematical definitions. This could lead to incorrect results, and the pre-registered support condition was not unambiguously met. | next: Develop a more accurate implementation of the geometric invariant group order and circuit monotone width calculations before further testing.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	19595
2	preregistration	ollama_remote	glm4:latest	0	0	9759
3	novelty	ollama_remote	glm4:latest	0	0	10819
4	novelty	ollama_remote	glm4:latest	0	0	23450
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13806
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17959
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8690
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7647
9	critic	ollama_remote	glm4:latest	0	0	11190
10	judge	ollama_remote	glm4:latest	0	0	9339

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 132253 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/f5c9b3eb3fcd.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f5c9b3eb3fcd.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/f5c9b3eb3fcd.tar.gz` (if generated)